Developing a novel methodology for analysing egress utilising graph theory applied to IFC

Richard Lai

Master of Engineering Science
Fire Safety Engineering

The University *of* Queensland
2026

Developing a novel methodology for analysing egress utilising graph theory applied to IFC

by Richard Lai

Student Number: **49568479**
Course Code: **CIVL7513**
Supervisor: **Dr David Lange & Dr Anwar Orabi**
Submission Date: **08/06/2026**

School of Civil Engineering
The University of Queensland

Abstract

To be completed at the end of the project.

Acknowledgement of AI use

Tool	Use	Prompts	Section	Date
Perplexity	Refinement of objectives for the project that I adapted	Please refine the objectives I've listed to improve the language for a research thesis.	Section 1.2	16 May 2025
Microsoft Copilot	To find relevant literature that supports my claim of pathfinding applications	Find me a research paper that describes applications of A*	Section 2.5	30 May 2026
Microsoft Copilot	To receive feedback on whether my project background contextualises my problem statement	Review section 1 and tell me if it clearly defines the problem statement	Section 1.1	30 May 2026
Perplexity	Spelling and grammar check of report	Perform a spelling and grammar check of this report.	All	6 June 2026

Weekly Logbook

Course code: CIVL7513

Student ID: 49568479

Student Name: Richard Lai

Week	Activity, topics discussed, actions	Supervisor initials
1	Discussed rough overview of the research project, timelines, objectives and structure. Discussed potential topics and established general ideas around applications of BIM in fire engineering. Set goals to read literature to develop more specific ideas around the topic and define goals.	DL/AO
2	Reviewed some literature surrounding BIM use in industry, specifically to fire safety engineering. Discussed potential topics, relating to IFC schema and use of fire safety property sets for better integration into industry. Actions for the week were to continue reading into relevant literature and home in on research topic.	DL/AO
3	Presented findings from literature reviews, which included some topics on using BIM to enhance evacuation analysis and simulations. Discussed the opportunity to implement graph theory to represent the IFC node structure. Actions are to familiarise with graph theory, and begin looking into specific ways that information from IFC can be extracted to be used in fire engineering applications.	DL/AO
4	Discussed potential research objective, to extract parameters useful for fire modelling or analyse egress using IFC models. A number of useful tools to continue on this path were brought up during the session. Specifically, using IfcOpenShell as a way to code and extract information directly from an IFC model would be useful. Focus for the week is to become more familiarised with IFC structure and read documentation on IfcOpenShell.	DL/AO
5	Explored IFC documentation to gain familiarity with the structure and nodes.	DL/AO

	Presented findings from IfcOpenShell documentation and tested commands from a sample mode. Further experimentation in extracting more information from an IFC model is proposed. Discussed aim and objectives for the thesis, and to attempt to establish a draft in the coming week.	
6	Experimented with extracting wall properties, openings and location coordinates of objects in an IFC model, using IfcOpenShell. Discussed Aim and Objectives that were established, focused around the developing an egress analysis strategy using IFC. Focus of the week is to construct geometry using IfcOpenShell, and BIM software to gain appreciation of how geometry creation is completed.	DL/AO
7	No meeting was had this week. Started installing BIM software, and created wall geometry using IfcOpenShell. Continue geometry creation using these two methods for the week.	DL/AO
8	Constructed a small building with two rooms, and a corridor using IfcOpenShell and Archicad. Discussed challenges in both, and also advantages. It was concluded that IfcOpenShell is favoured as there is better control over the IFC parameter outputs Goals for the week were to make a start on progress report by defining structure.	DL/AO
9	Developed structure for Progress Report with headings and subheadings. Brainstormed modifications to the current "Aim" of the thesis Created zones in IFC and investigated extracting information from these models. Additional progress in Progress Report to be made during the following week. In addition, create openings between rooms in the IFC model.	DL/AO
10	Refined Thesis Title and discussed the wording surrounding it. Began writing out sections of the progress report Created openings between rooms in the IFC model. Also drew relationships within the code to relate rooms to boundary spaces via the openings	DL/AO

	Further research into algorithms used for the project to be done.	
11	Drafted introduction sections of the progress report. Also, established that A* can be used as a pathfinding algorithm for the methodology to evaluate travel distance in the IFC model. Discussed code semantics and QoL refinements such as naming conventions. Actions are to continue work on Progress Report.	DL/AO
12	No meeting this week as RL was busy. Aims and objectives were reviewed and refined. Continued work on progress report.	DL/AO
13	No meeting as DL and AO were busy. Continued work on progress report for submission at the end of semester.	DL/AO

Table of Contents

1. Introduction 9
1.1 Project Background 9
1.2 Research Aims and Objectives 10
2. Literature Review 11
2.1 BIM in Context 11
2.2 Examples of Evacuation and Pathfinder in BIM 11
2.3 IfcOpenShell 11
2.4 Graph theory 12
2.5 Pathfinding 13
3. Methodology 14
3.1 Overview 14
3.2 Model Creation IfcOpenShell 14
3.3 Extracting Parameters from Model 15
3.4 Algorithm to find longest paths of travel 15
3.5 Validation using Pathfinder 16
4. Preliminary Results 17
4.1 Overview 17
4.2 Model Creation 17
4.3 Using Openings to Establish IfcSpace Relationships 18
4.4 Full Graph Network 19
4.5 Parameter Extraction 20
5. Risk and Project Management 22
5.1 Project Timeline 22
5.2 Risk Management 22
5.2.1. Scalability and Time Complexity 22
5.2.2. Validation and IFC Compatibility 22
5.2.3. Code Debugging and Delays 22
6. Preliminary Conclusion 23
7. Bibliography 24
Appendix A 25

Table of Figures

Figure 1 LPG-based graphs for Road 701 - Fig. 8 [8] 12
Figure 2 Internal graph structure for relational information Fig. 9 [8] 12
Figure 3 Pseudocode for A* algorithm [11] 16
Figure 4 Sample building made with IfcOpenShell geometry creation 17
Figure 5 Python function to create a room in IfcOpenShell 18
Figure 6 Simple example of a room connected to a corridor represented as a graph 19
Figure 7 Visualisation of the graph of the IFC building generated 19
Figure 8 Python code for parameter extraction functions 20

1. Introduction

1.1 Project Background

The use of Building Information Modelling (BIM) is becoming increasingly prevalent within the construction industry. The advantages of BIM are being recognised as it contains the powerful characteristic of containing information within the building models itself. For example, a particular wall in a BIM project can contain information about its material layers, wall thickness, geometrical placement, and identified by its own unique ID. Industry Foundation Classes (IFC), developed by buildingSMART, is the open international standard for BIM data and is published as ISO 16739. IFC is a typical data structure used to combine traditional CAD models with the capability of description.

Several disciplines within the construction industry have already adopted the IFC format. Common procedures involve Architecture leading the project by creating the base model of the building, and technical disciplines following through. For example, a Hydraulics Engineer would use the Architectural base to design where their services equipment needs to be located. They can then create their own models such as pipework and insert it into the overall BIM project to line up with the existing Architectural design. However, Fire Safety Engineers have not yet effectively adapted to using IFC.

The lack of integration with IFC for Fire Safety Engineers could be for a few reasons. Firstly, it is often that they do not have any tangible contributions to the model that are not already captured by other disciplines. Suppression systems and fire hydrants will be designed and implemented by the Fire Services Engineers, or Hydraulics Engineers, therefore there is no input required.

In the absence of creating a model, Fire Engineers may be able to modify or extract information from an existing model. By working together with the design team, a new system can be developed to define objects such as fire walls and fire doors. This could give designers greater oversight of the fire safety systems of the building, however it would also require a large overhaul of the current standardised methods to implement fire safety systems within IFC. This would also need to be supported through common industry-used software such as Autodesk Revit.

Current applications of IFC are used in simulation tools such as Thunderhead's Pathfinder and Pyrosim. These software allow import of IFC files, such that the layout of the building can be directly input. However, these imports are not perfect as there can be issues such as disconnected floors and overlapping objects in Pathfinder. And in Pyrosim, the sheer number of objects that make up the building in an IFC model tends to be overwhelming and reduces the software's performance.

The issues mentioned above are only some of the challenges that Fire Engineers face when working with IFC, that often become a barrier that prevents further pursuit and upskilling in this area. Further divergence away from IFC may lead to increased isolation of Fire Engineers from the design team. It is clear that Fire Engineers need to be involved to help drive the design especially from an early stage. Therefore, increased understanding and utilisation of IFC will enhance integration with other designers and potentially present more opportunities to be involved in the design.

This gap in IFC involvement has been identified, and it is recognised that significant progress has been made in bridging this gap [1]. However there has been limited practice in extracting the information from an IFC model and repurposing this for Fire Engineering tasks and analysis. This information extraction can be taken advantage of, for even more trivial tasks. Hence, there is a motive for this project to explore new methods of utilising BIM directly for everyday Fire Engineering problems. Further research into these methods may provide the stepping stones required to progress industry Fire Engineering involvement within the BIM environment.

1.2 Research Aims and Objectives

This project will focus on one potential use of IFC, namely information extraction, and apply it to a common problem seen throughout Fire Safety Engineering. The intent is to bring the spotlight to the use of IFC through alternative methods and establish a framework that can be developed for future applications.

Therefore, the aim of this project is to develop and validate a novel graph-based methodology for analysing egress within buildings by deriving travel distances from IFC models using IfcOpenShell.

The objectives undertaken for this project to realise the aim are as follows:

- Establish an IFC model that produces a structured representation of a building consisting of walls, doors and rooms suitable for automated egress analysis using IfcOpenShell.
- Formulate a graph-based representation of the building egress network by abstracting IfcSpaces and IfcOpenings into nodes and edges of a graph.
- Develop and implement edge weightings utilising straight-line geometry distances that enable computation of combined egress paths within the IFC graph.
- Apply and evaluate pathfinding algorithms such as A* for identifying critical egress paths within the building.
- Validate the proposed graph-based egress methodology by comparing the derived travel paths and calculated distances with results from Pathfinder evacuation simulations, and to assess its suitability for egress or travel distance computations.

2. Literature Review

2.1 BIM in Context

With the development of digital information, BIM is becoming more common in industry practices. The research paper in [2] highlights the prevalence of fire safety engineering in BIM, through a systematic study of 106 papers. The research also found that, over the past decade, fire and evacuation simulations have made up most of the reviewed papers, using a range of mathematical models and technologies. These methodologies often involve complex integrations within BIM, for example, to find the shortest path of travel. It also found that of these papers, many of the tools relied on a higher Level of Development (LoD) [3]. Therefore, a building in its early design phase, which has only been modelled to a basic LoD cannot utilise the potentially unique parameters that are required to execute the Fire Safety BIM simulation. This circles back to the topic of the adaptability of Fire Safety Engineering in the digital environment, and concerns for practical integration in industry.

2.2 Examples of Evacuation and Pathfinder in BIM

A practical example of Fire Engineering integration is through the use of evacuation modelling tools such as Pathfinder [4]. This software is capable of importing IFC files to generate the geometry required for the simulation. In concept, this method is straightforward, however in practice still may require manual re-building of the model within Pathfinder, due to inclusion of additional and unnecessary objects from the extracted geometry. Thunderhead Engineering also acknowledges these potential issues within their own import guides [5].

The research in [6] proposes a framework for two-way data sharing between Pathfinder and Autodesk Revit, the latter being a common software used to develop buildings using BIM. In this paper, it is suggested to develop an add-in Evac4BIM, which defines an additional list of data properties in the IFC model. The benefit being that there is a more seamless connection and compatibility between the two programs, and even allowing evacuation results to be presented within the Revit model. This process requires global adoption from designers and standardisations that incorporate the Fire Engineering add-in for better interoperability.

For these applications, it is evident that even simple evacuation modelling can have complicated interactions in BIM that also require additional support or third-party intervention. Despite this, existing research demonstrates the importance and benefits of applying these tools in BIM. However, increased complexity may worsen the barrier of entry for Fire Safety Engineering and BIM. Hence, an alternative method for integration that uses non-standard practices are sought.

2.3 IfcOpenShell

When speaking about BIM, the term IFC is usually mentioned. Developed by buildingSMART [7], IFC is the structured information file that Fire Engineering software such as Pathfinder and Pyrosim can import for direct model generation. The file is usually generated as an exported file from a modelling software, such as Revit, and contains all information defined from the model. IfcOpenShell is an open-source toolkit specifically for IFC, which defines methods for a user to extract any information from the model. Conversely, it can also be used for geometry generation, however use of IfcOpenShell is not common practice in lieu of BIM

modelling software. IfcOpenShell does however have potential for use in extracting simple building parameters from a model relatively quickly.

2.4 Graph theory

A structured information model like IFC is naturally represented by a mathematical graph. The IFC structure consists of multiple properties, also considered as nodes, that are all connected in some way within the model. From the journal article in [8], Fig. 8 shows the entire graph structure within an example IFC model of a road, with decreasing LoD of the model. This illustrates the vast interconnection of information within IFC.

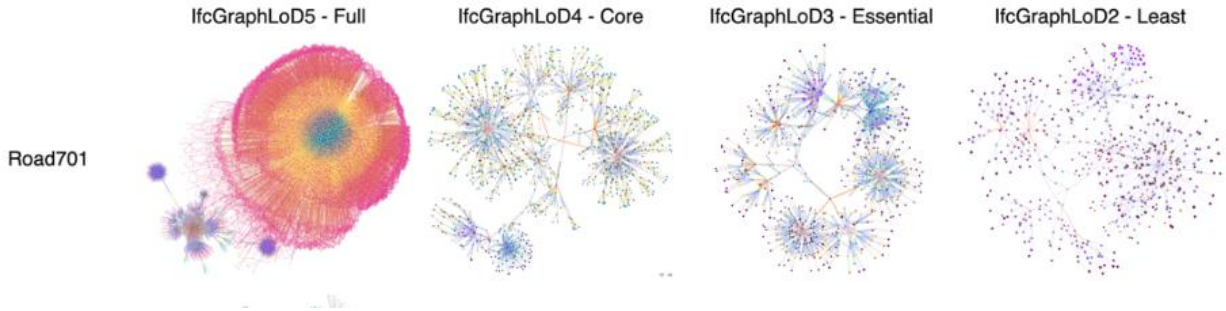


Figure 1 LPG-based graphs for Road 701 - Fig. 8 [8]

Within the same journal article, an internal graph structure of a simple house is also shown that describes only the spatial relationships.

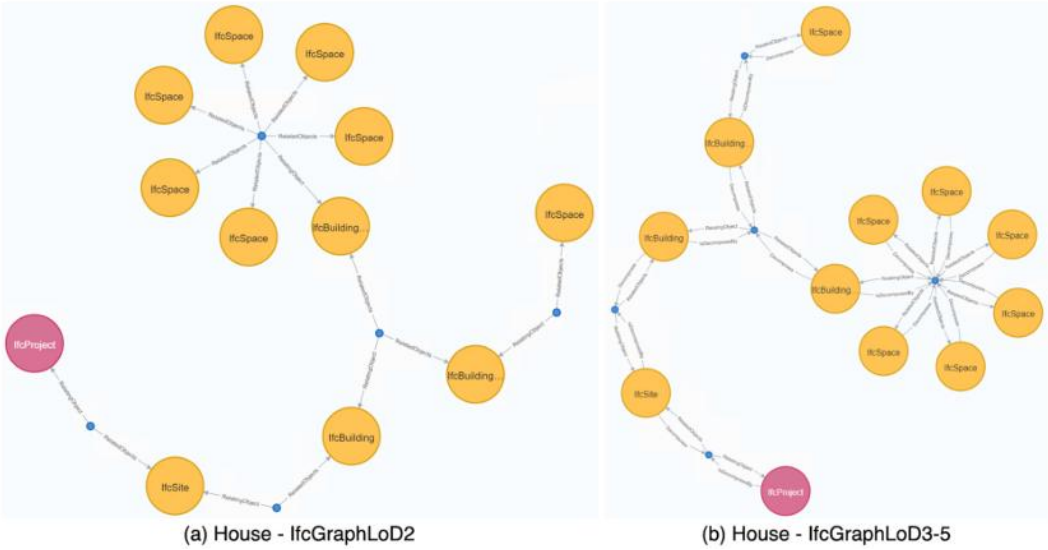


Figure 2 Internal graph structure for relational information Fig. 9 [8]

These figures illustrate the natural relationship between graphs and the IFC structure.

Research done in [9] implements a robot navigation model that is based on BIM and IFC. Using similar concepts, a graph representation of a floor layout is used to conduct a navigation algorithm. The graphs make use of simple wall geometry and doors which define the layout. *IfcSpace* elements are used to distinguish between rooms, which can then be used to calculate travel paths within a building. In doing this, the problem reduces the 2D navigation problem to a graph traversal problem, which can be easily manipulated.

Although this is a navigation model for path planning, similarities can be drawn to modelling evacuation strategies for a building.

2.5 Pathfinding

In many fields of study, path planning algorithms make use of the A* (A-star) algorithm in graph-based search methods. The paper in [10] presents a summary of the research completed on A*. This method, based on evaluation of the lowest cost of any path, is simply represented by the following equation:

$$f(n) = g(n) + h(n)$$

Where $f(n)$ is the cost function, $g(n)$ is the actual cost to the current node from the starting node, and $h(n)$ is the heuristic function.

Big O notation is used to classify how an algorithm's computational time may scale with its input. In general, the time complexity of the A* algorithm can be up to $O(b^d)$, where b is the branching factor, and d is the solution depth [11]. For a fixed and finite graph, this is equivalent to $O(ElogV)$, where E is the number of edges and V are the number of vertices of a graph. It is noted that this is a worst possible case for the time complexity, and it can be reduced depending on the choice of heuristic

Common heuristic functions used are based on Manhattan Distance, Euclidean Distance, and Chebyshev Distance. These are described in the table below.

Table 1 Common heuristic functions used in A and their definitions*

Heuristic Function	$\boldsymbol{h(n)}$
Manhattan Distance	$\lvert x_1 - x_2\rvert + \lvert y_1 - y_2\rvert$
Euclidean Distance	$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
Chebyshev Distance	$\max(\lvert x_1 - x_2\rvert, \lvert y_1 - y_2\rvert)$

The effectiveness of A* relies on appropriate selection of the heuristic function, which is typically an estimate of the actual cost. Kuznetsov also describes practical applications of A* in [11]. These include robot navigation, game development and route planning. Given its predominant use in several different applications, A* is also considered to be an appropriate tool to utilise for analysing egress scenarios within a graph-based IFC building.

3. Methodology

3.1 Overview

The project will consist of developing an IFC building model in Python through the use of the Python library IfcOpenShell, then using this model to extract information required for the method. IfcOpenShell is used to construct the geometry, but more importantly to set up the structure of the information modelling. The model structure is developed specifically such that the information can be extracted consistently. Such information needed to be extracted is *IfcSpace* boundary relationships, object locations, or if any openings exist within a room. Some properties may be retrieved or defined in a model in different ways, depending on how the user or software has configured it. Hence, a model has been developed in IfcOpenShell for consistency in information extraction for this project.

Once the building model has been created, additional functions are developed to extract different parameters from the model. These parameters are then used for input into the pathfinding algorithm. The pathfinding algorithm will determine the longest paths of travel from any point of the building to the exit, simply by input of the IFC model.

Finally, Pathfinder will be used as a validation method. The IFC model that was constructed will be imported into Pathfinder as a basis for the model geometry. Simulations will be performed on the geometry to assess the travel distances as a benchmark for the algorithm developed. Further details of each method are described in this section.

3.2 Model Creation IfcOpenShell

The model for the building is constructed more manually using IfcOpenShell, as opposed to more traditional methods such as Autodesk Revit, or Archicad. The reasoning behind this, as mentioned earlier in Section 3.1, is due to potential inconsistencies of IFC models generated between different platforms. That is, different software could store the IFC information about the specific building parameters in different locations, or require an entirely different calling function to access this information. Therefore by using IfcOpenShell and then specifically selecting the way parameters are accessed allows for a consistent model structure. This can then be more easily utilised to achieve the main objectives and aim of this research project.

IfcOpenShell is an open source library that is compatible with Python. The model is created using code in Python and relevant commands defined by IfcOpenShell. Some functions are created in Python to allow for easier creation of the geometry in IFC. These functions are to create a slab, wall, door and space. A space in this instance is referred to as an *IfcSpace*, which is used to distinguish between different rooms within a storey. Therefore, there will be multiple *IfcSpace* entities within a storey, where each *IfcSpace* may be connected to another via an opening (i.e. door). This relationship is defined through another function to define *IfcSpace* boundaries and relationships. The primary functions in Python used for creating geometry are described in Table 2.

Table 2 Description of functions defined to generate building entities to the IFC file

Function	Description
create_slab	Creates a slab of specified length and width at the selected location.

create_wall	Creates a wall of specified length and height at the selected location.
create_space	Creates an *IfcSpace* of specified length and width at a selected location. Additionally, it creates a new child element "*IfcSpace*" for the subject storey.
create_space_boundary	Creates a virtual boundary entity of class "*IfcRelSpaceBoundary2ndLevel*" relating the specified *IfcSpace* to the specified opening.
create_opening	Creates an opening of specified width and height by placing a void entity within a selected wall and location. Additionally, relates the opening to the *IfcSpace* of the connected room/s.
create_room	Performs *create_slab*, *create_wall*, and *create_space* to create a room with geometry and defined IfcSpace of specific length and width at the selected location.

3.3 Extracting Parameters from Model

In order to obtain information about the IFC model, additional functions are created to extract this information from the model. This information will be used as input to the pathfinding algorithm described in Section 3.4. At this stage of the project, the information required to complete the task is not confirmed, as the algorithm is not yet completely defined. However, it is expected that functions to extract the coordinates of objects and determining adjacent boundary rooms (i.e. adjacent *IfcSpace*) will be necessary.

Further details will be confirmed at a later stage of the project.

3.4 Algorithm to find longest paths of travel

Once the IFC building model is created, the geometry will be set and so will the network of *IfcSpace* entities. The *IfcSpace* elements are connected to each other via the door openings. Therefore the entire structure of the building can be represented as a graph network. The openings in the model will be represented by the nodes in this graph, and edges will represent the travel path to another opening within a room. The intermediate travel distance between rooms will be the corresponding weighting for traversal via edges. Using this representation of the IFC model, the A* method will determine where the longest travel distances are located from any point of the building. This will be repeated for various starting points throughout the building to determine where the longest paths of travel occur.

The A* method, as described in Section 2.5, evaluates the cost function and advances to next node by choosing the lower cost path. As mentioned, this method relies on a good heuristic function.

Pseudocode for the A* algorithm is provided from [11]. This pseudocode is copied below in Figure 3 for reference.

```
def a_star(start, goal, h):
  open_set = {start}
# Nodes to be evaluated
  closed_set = {}
# Already evaluated nodes
  g_score = {start: 0}  # Cost from start to current node
  f_score = {start: h(start)}  # Estimated total cost
  while open_set is not empty:
      current = node in open_set with lowest f_score
      if current == goal:
          return reconstruct_path(current)
      open_set.remove(current)
      closed_set.add(current)
      for neighbor in get_neighbors(current):
          if neighbor in closed_set:
              continue
          tentative_g = g_score[current] + cost(current, neighbor)
          if neighbor not in open_set:
              open_set.add(neighbor)
          elif tentative_g >= g_score[neighbor]:
              continue
          g_score[neighbor] = tentative_g
          f_score[neighbor] = g_score[neighbor] + h(neighbor)
```

Figure 3 Pseudocode for A algorithm [11]*

Further details of this procedure specific to this topic will be developed and completed at a later stage of the project.

3.5 Validation using Pathfinder

Upon completion of the steps outlined above, the longest travel distances will be validated through Pathfinder. The IFC model created will be imported into Pathfinder for geometry creation, then used to determine the egress routes in the simulation. Firstly, occupants will be placed in each room. Then the simulation will determine which occupants, and therefore which locations take the longest amount of time to evacuate the building.

4. Preliminary Results

4.1 Overview

The majority of the progress to date has been within programming using Python and experimenting with geometry creation with IfcOpenShell. The sections below describe parts of the progress to date. Some examples of the code are provided for context and demonstration purposes. This section presents preliminary results of the model creation, parameter extraction, and visualisation of graph networks using programming tools.

However, it is acknowledged that presenting the entire code base within a report is impractical. Therefore, the source code can be provided upon request, with the acknowledgement that the code is still in development, and current progress to date is purely for demonstration purposes.

4.2 Model Creation

A relatively simple, single storey building has been constructed using IfcOpenShell. The building consists of 14 rooms, 13 internal openings and 3 exits. The geometry of the IFC model is shown in Figure 4.

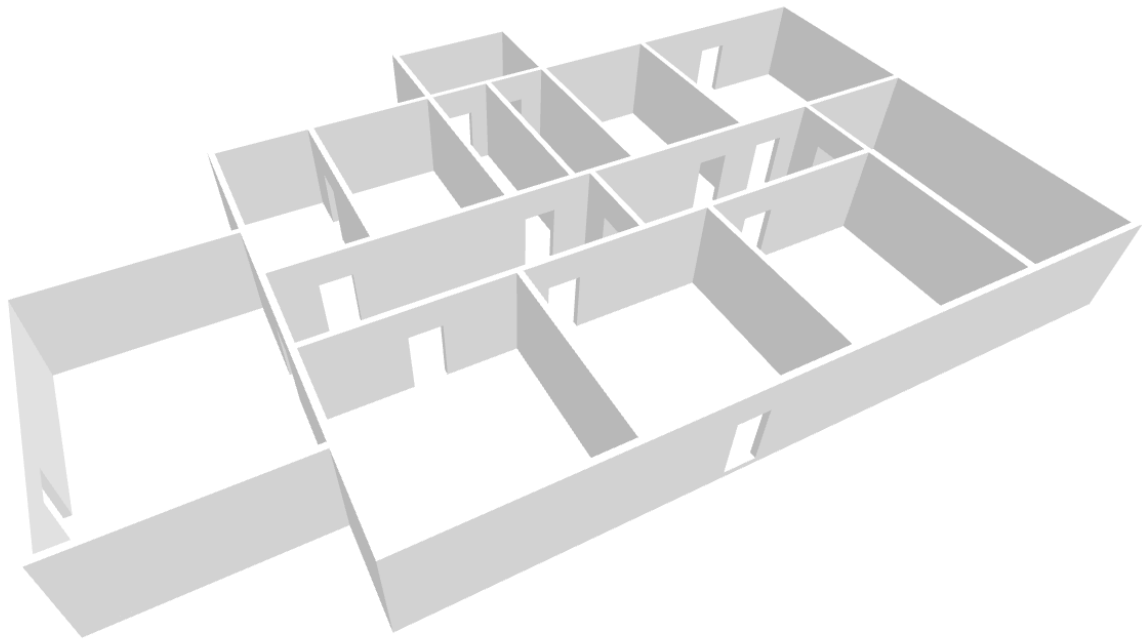


Figure 4 Sample building made with IfcOpenShell geometry creation

The geometry was created room by room, by using the functions as described in 3.2. The function "*create_room"* is shown in Figure 5.

```
# define a function to add a room to the model, which is just an ifcspace with
physical geometry assigned
def create_room(length, width, name, origin):

    room_dict = {}

    [ifcspace, space] = create_space(length=length, width=width, name=name,
origin=origin+np.array([length/2, width/2, 0]))

    room_dict[name+"wallN"] = create_wall(length=length, name=name+"wallN",
origin=origin+np.array([0, width, 0]), rotate=False, ifcspace=ifcspace)
    room_dict[name+"wallS"] = create_wall(length=length, name=name+"wallS",
origin=origin+np.array([0,-0.2,0]), rotate=False, ifcspace=ifcspace)

    room_dict[name+"wallE"] = create_wall(length=width+0.4, name=name+"wallE",
origin=origin+np.array([length+0.2, -0.2, 0]), rotate=True, ifcspace=ifcspace)
    room_dict[name+"wallW"] = create_wall(length=width+0.4, name=name+"wallW",
origin=origin+np.array([0, -0.2, 0]), rotate=True, ifcspace=ifcspace)

    room_dict[name+"slab"] = create_slab(length=length+0.4, width=width+0.4,
name=name+"slab", origin=origin+np.array([length/2, width/2, -0.2]),
ifcspace=ifcspace)

    return [space, room_dict]
```

Figure 5 Python function to create a room in IfcOpenShell

This function creates both virtual room and physical room geometry using the inputs firstly by defining an *IfcSpace* element and then creating individual wall and slab geometry within this space. Below is an example of using this function to create a room.

$$Room1 = add_room(length = 5, width = 4, name =' Room1", origin = (0,0,0))$$

This creates a rectangular room called “Room1” with a length of $5\ m$ and width of $4\ m$ centred about the global axis with coordinates $(0, 0, 0)$.

“Room1” then returns an *IfcSpace* element located within those coordinates and a dictionary containing the corresponding *IfcWall* and *IfcSlab* elements. The walls and slabs are created using functions “*create_wall*” and “*create_slab*”, respectively.

4.3 Using Openings to Establish IfcSpace Relationships

Openings are created within the walls of a defined room, or two rooms if the wall is common to both. In addition to being a physical geometric opening, it is imperative that this opening creates a relationship between the two rooms, which are classed as *IfcSpace* elements. The property, *ifc_class* for this relationship entity must be established as “*IfcRelSpaceBoundary2ndLevel*”. As a simple example, a room in a building may lead to a corridor via a door. When represented as a graph, this is illustrated below.

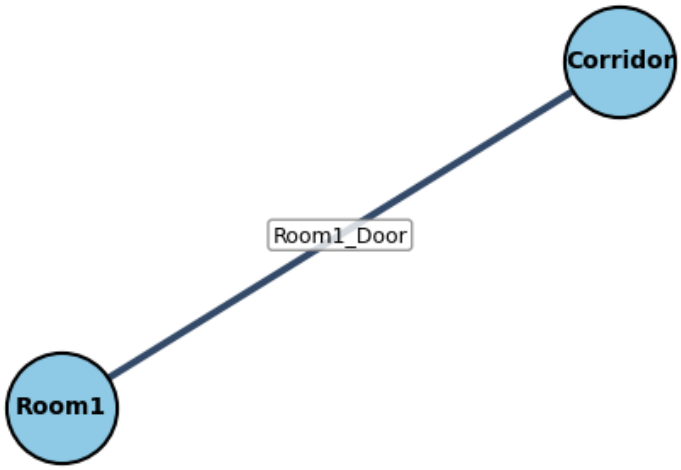


Figure 6 Simple example of a room connected to a corridor represented as a graph

In the figure, "Room1" and "Corridor" are represented as nodes of the graph, where "Room1_Door" defines the relationship between them, therefore constructing the edge. This concept is followed to construct the remainder of the IFC file and develop the graph.

4.4 Full Graph Network

To construct and visualise the IFC graph, additional Python libraries are used. These are *networkx* and *matplotlib*. As discussed, *IfcSpace* entities will represent the nodes, and *IfcOpeningElement* entities will represent the edges which connect these nodes. For the building illustrated in Figure 4, the graph for this preliminary model is shown below.

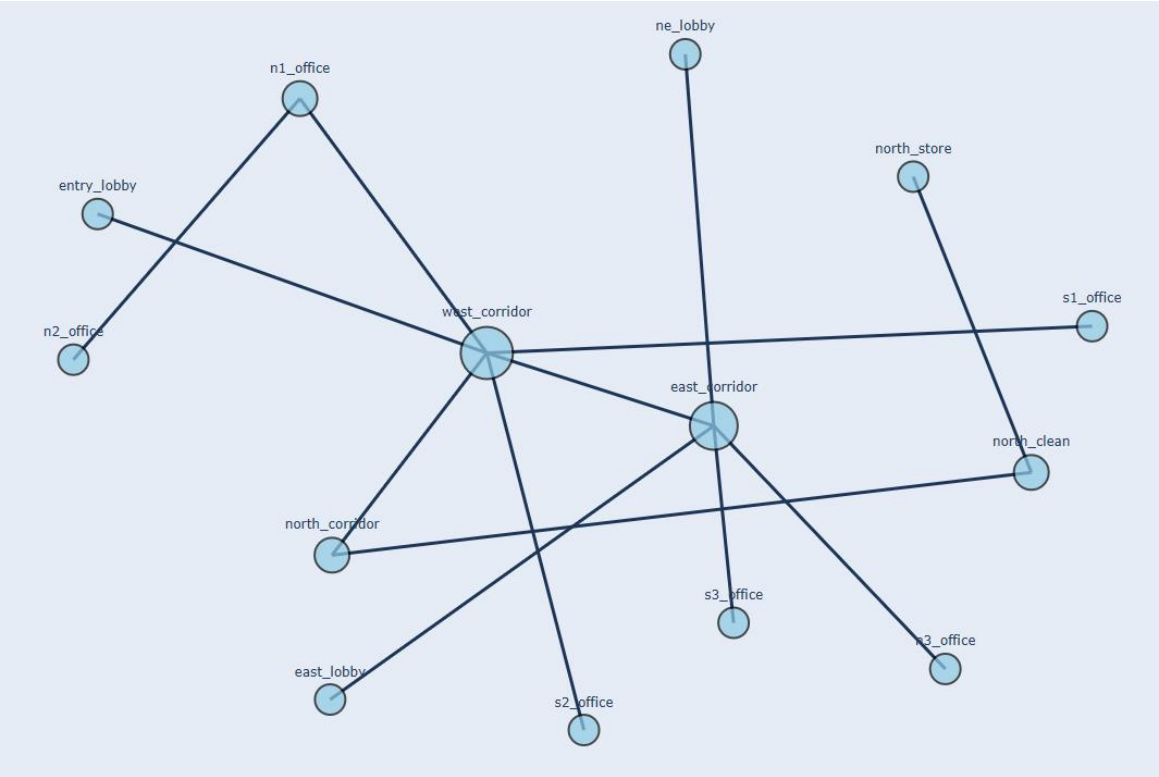


Figure 7 Visualisation of the graph of the IFC building generated

Future development of the design will incorporate a larger model, therefore a larger graph, and include weightings on the edges. These weightings will be based on travel distance between rooms and will represent the cost factors in the A* methodology.

4.5 Parameter Extraction

At this stage, three functions have been made for extracting parameters from the IFC. These are described in the table below.

Table 3 Description of functions defined to extract parameters from IFC file

Function	Description
get_boundary_room	Returns a dictionary of *IfcSpace* entities adjacent to the selected *IfcSpace* (room)
get_openings	Returns a dictionary of *IfcOpeningElement* entities (openings) contained within the selected *IfcSpace* (room)
get_coordinates	Returns the origin of the selected element or entity in its geometric cartesian coordinates (x,y,z)

To demonstrate the simplicity of these functions, the Python code is provided below.

```
def get_boundary_room(space):
    b_room = {}
    for boundary in space.BoundedBy:
        corr_boundary = boundary.CorrespondingBoundary
        b_room[boundary.RelatedBuildingElement.Name] =
corr_boundary.RelatingSpace
    return b_room

def get_openings(space):
    openings = {}
    for boundary in space.BoundedBy:
        if boundary.RelatedBuildingElement.is_a("IfcOpeningElement"):
            openings[boundary.RelatedBuildingElement.Name] =
boundary.RelatedBuildingElement
    return openings

def get_coordinates(entity):
    matrix =
ifcopenshell.util.placement.get_local_placement(entity.ObjectPlacement)
    return matrix[:,3][0:3]
```

Figure 8 Python code for parameter extraction functions

These functions utilise simple commands as part of the IfcOpenShell package to obtain certain parameters. As the model has been defined as a connected graph, it is simple to traverse entity relationships to obtain relevant information about the building.

These functions may be refined, and additional functions will be developed as the project progresses.

5. Risk and Project Management

5.1 Project Timeline

For the organisation of the project, the key tasks required to progress the project have been developed and scheduled in a Gantt chart. The timeline of tasks is only indicative of the time required for completion but serves to provide a flexible framework for planning.

The timeline is shown in Appendix A.

5.2 Risk Management

5.2.1. Scalability and Time Complexity

A relevant risk that has been identified in the project is related to scalability. As currently progressed, the model constructed is relatively small. When a larger building is created as part of the project timeline, the impact of the larger model could pose a risk. It is noted that the algorithm for pathfinding in this model has not been completely prepared yet, however the estimated time complexity can be up to $O(ElogV)$. Therefore, in theory, scalability is not expected to be a significant issue for the size of the building, however, it should be acknowledged. It should also be noted that even with the expansion of the building, the size of the graph is still expected to be relatively small, in comparison to other significant applications of the A* algorithm.

5.2.2. Validation and IFC Compatibility

Another risk to the project is within the validation process. The procedure for this task is to import the generated IFC model into Pathfinder to develop the floors for use in the egress simulation. However, it is noted that the IFC model was not developed using traditional methods, such as export through standard software. Therefore, there is a risk of incompatibility of importing the model into Pathfinder, which could cause disruptions and delays to the validation task. Based on the timeline of this project, the objective is to complete the final modelling and calculations required relatively early in the semester. This allows more time for documentation of the thesis but also is intended to allow for a buffer when completing tasks.

5.2.3. Code Debugging and Delays

Additionally, as the project involves a substantial amount of programming using Python, risks related to errors and the debugging process must be taken into consideration. This also alludes to flow on effects, as each project task has dependencies on the previous task. Therefore, if delays occur due to debugging, this may cause compounding delays leading to a rushed program. The project timeline employed aims to mitigate these risks by following through an indicative guideline for completion of tasks.

6. Preliminary Conclusion

The progress report highlights the key research and development of the project to date. During the literature review, the current capabilities for Fire Safety Engineers to collaborate with BIM models were investigated to determine where the industry sits, and where opportunities of research are most valuable. In doing so, this preliminary study has proposed to pursue a novel methodology to evaluate a rather common problem for Fire Engineers. This research aims to provide insight into the industry and prepare frameworks for future innovative experiments.

The current work has produced a simple building using IfcOpenShell commands and defining repeatable functions to construct a building. An IFC file has also been generated for the building with appropriate node structure and relationships. This IFC file has been processed and constructed as a graph, which will become the framework for the algorithm development in the next semester.

7. Bibliography

[1] S. Bigdeli, *The benefits and challenges of integrating BIM into fire safety design,* Jensen Hughes, 2022.

[2] A. C. M. L. &. G. Z. Ada Malagnina, “The Digital Transformation in Fire Safety Engineering over the Past Decade Through Building Information Modelling: A Review,” *Fire Technology,* vol. 58, pp. 3317-3351, 2022.

[3] BIMForum, “Level of Development Specification Part 1,” BIMForum, 2024.

[4] Thunderhead Engineering, “Pathfinder | Crowd Movement Simulation and Egress Software,” 2026. [Online]. Available: https://www.thunderheadeng.com/pathfinder/.

[5] Thunderhead Engineering, “IFC Import to Pathfinder (and PyroSim,” 2026. [Online]. Available: https://www.thunderheadeng.com/docs/2026-1/pathfinder/examples/how-to/ifc-import/.

[6] P. T. A. S. J. A. &. E. R. Nazim Yakhou, “The integration of building information modelling and fire,” *Journal of Building Engineering,* vol. 63, 2023.

[7] buildingSMART, “Industry Foundation Classes (IFC),” [Online]. Available: https://www.buildingsmart.org/standards/bsi-standards/industry-foundation-classes/.

[8] N. N. R. K. Y. W. M. W. I. B. Junxiang Zhu, “Revealing the internal structure of IFC-Graph for efficient querying and,” *Advanced Engineering Informatics,* vol. 70, 2026.

[9] G. Ś. B. S. &. E. G. Wojciech Palacz, “Indoor Robot Navigation Using Graph,” in *Lectures Note in Computer Science*, Springer Nature Switzerland, 2019, pp. 654-665.

[10] J. M. F. P. H. L. Dongke Ran, “Summary of Research on Path Planning Based on A* Algorithm,” *The 8th International Symposium on Test Automation & Instrumentation,* pp. 84-89, 2020.

[11] O. Kuznetsov, “The A* Algorithm,” in *Intelligent Systems: From Theory to Applications*, Springer Cham, 2026, pp. 167-188.

Appendix A

	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	BREAK	Week 10	Week 11	Week 12	Week 13	SWOTVAC	
Tasks	27/07/2027	3/08/2027	10/08/2027	17/08/2027	24/08/2027	31/08/2027	7/09/2027	14/09/2027	21/09/2027	28/09/2027	5/10/2027	12/10/2027	19/10/2027	26/10/2027	2/11/2027	9/11/2027
Deliverables													**Presentation Slides: 20/10/2026** **Final Presentation: 21/10/2026-23/10/2026**			**Final Report: 09/11/2026**
IFC geometry creation																
Develop IFC graph with weighted edges																
Develop A* algorithm for IFC graph																
Automate the identification and output of long paths of travel																
Pathfinder Validation																
Results Analysis and Documentation																
Presentation Preparation																
Thesis Documentation																

Legend
Project development & coding
Deliverables documentation
Submission dates